16

Сегментация

До сих пор мы помещали все адресное пространство каждого процесса в память. С помощью регистров base и bounds ОС может легко перемещать процессы в различные части физической памяти. Однако, возможно, вы заметили что-то интересное в этих наших адресных пространствах: между стеком и кучей есть большой кусок “свободного” пространства прямо посередине.

Как вы можете себе представить из рисунка 16.1, хотя пространство между стеком и кучей не используется процессом, оно все еще занимает физическую память, когда мы перемещаем все адресное пространство где-то в физической памяти; таким образом, простой подход использования пары регистров base и bounds для виртуализации памяти расточителен. Это также делает довольно трудным запуск программы, когда все адресное пространство не вписывается в память; таким образом, база и границы не так гибки, как хотелось бы. И таким образом:

Как поддерживать большое адресное пространство

Как мы поддерживаем большое адресное пространство с (потенциально) большим количеством свободного пространства между стеком и кучей? Обратите внимание, что в наших примерах с крошечными (притворными) адресными пространствами отходы не кажутся слишком плохими. Представьте, однако, 32-разрядное адресное пространство (4 ГБ); типичная программа будет использовать только мегабайты памяти, но по-прежнему будет требовать, чтобы все адресное пространство было в памяти.

16.1 Сегментация: обобщенная база / границы

Для решения этой проблемы родилась идея, которая называется **сегментацией**. Это довольно старая идея, идущая, по крайней мере, еще в самом начале 1960-х годов. Идея проста: вместо того, чтобы иметь только одну пару базы и границ в нашей MMU, почему бы не иметь пару базы и границ на каждый логический **сегмент** адресного пространства? Сегмент — это просто смежная часть адресного пространства определенной длины, и в нашем каноническом

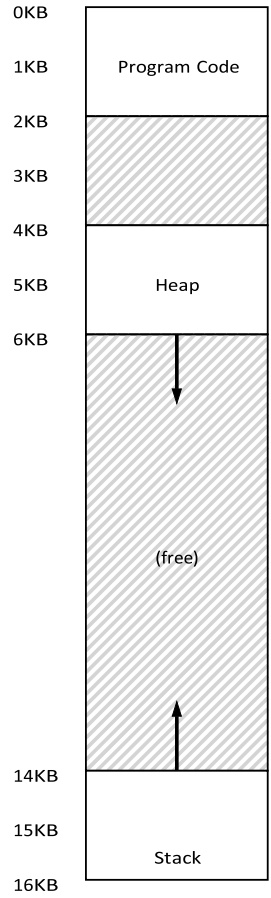


Рисунок 16.1: **Адресное пространство**

адресном пространстве, у нас есть три логически разных сегмента: код, стек и куча. Сегментация позволяет ОС размещать каждый из этих сегментов в разных частях физической памяти и таким образом избегать заполнения физической памяти неиспользуемым виртуальным адресным пространством.

Давайте рассмотрим пример. Предположим, мы хотим поместить адресное пространство из рисунка 16.1 в физическую память. С помощью пары база и границы на сегмент, мы можем разместить каждый сегмент независимо друг от друга в физической памяти. Например, см. Рисунок 16.2 (стр. 3); там вы видите физическую память 64 кб с этими тремя сегментами (и 16 КБ, зарезервированными для ОС).

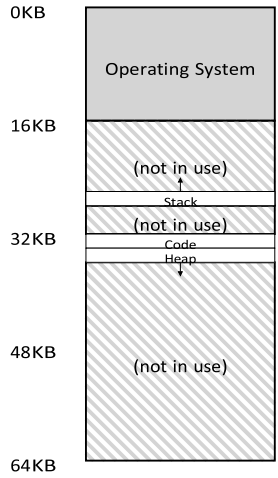


Рис. 16.2: **Размещение сегментов в физической памяти**

Как вы можете видеть на диаграмме, только используемая память выделяется в физической памяти, и, таким образом, в физической памяти большие адресные пространства с большим количеством неиспользуемого адресного пространства (которое мы иногда называем **разреженными адресными пространствами**) могут быть использованы.

Аппаратная структура в нашем MMU, необходимая для поддержки сегментации: в этом случае набор из трех пар базовых и граничных регистров. Рисунок 16.3 ниже показывает значения регистра для примера выше; каждый регистр границ содержит размер сегмента.

Segment	Base	Size
Code	32K	2K
Heap	34K	2K
Stack	28K	2K

Рис. 16.3: **Значения регистра сегмента**

Из рисунка видно, что сегмент кода расположен по физическому адресу 32КБ и имеет размер 2Кб, а сегмент кучи - 34КБ, а также имеет размер 2Кб.

Давайте сделаем пример перевода, используя адресное пространство на рисунке 16.1. Предположим, что ссылка сделана на виртуальный адрес 100 (который находится в сегменте кода). Когда ссылка имеет место (скажем, на выборке инструкции), оборудование добавит базовое значение к смещению в этот сегмент (100 в этом случае), чтобы прибыть в желаемый физический адрес: 100 + 32KB, или 32868. Потом проверит, что адрес

2008–18, ARPACI-DUSSEAU ©

находится в пределах границ (100 меньше, чем 2KB), находит то, что было по этому адресу, и выдает ссылку на адрес физической памяти 32868.

Неисправность сегментации.
Термин ошибка сегментации или нарушение сегментации возникает из-за доступа к памяти на сегментированной машине на недопустимый адрес.

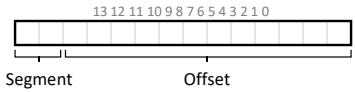
Теперь давайте посмотрим на адрес в куче, виртуальный адрес 4200 (опять же см. Рисунок 16.1). Если мы просто добавим виртуальный адрес 4200 к основанию кучи (34KB), мы получим физический адрес 39016, который не является правильным физическим адресом. Что нам нужно сделать, это извлечь **смещение в куче**, т. е. определить на какой/какие байт(ы) в этом сегменте ссылается адрес. Поскольку куча начинается с виртуального адреса 4KB (4096), то смещение 4200 — это фактически (4200 – 4096) = 104. Затем мы берем это смещение (104) и добавляем его к физическому адресу Базового регистра (34K), чтобы получить желаемый результат: 34920.

Что, если бы мы попытались обратиться к незаконному адресу, такому как 7KB, который находится за концом кучи? Вы можете себе представить, что произойдет: аппаратное обеспечение обнаруживает, что адрес находится за пределами границ, попадает к ОС, что, вероятно, приводит к прекращению процесса нарушителя. Это и есть **нарушение сегментации** или **ошибка сегментации.**

16.2 К какому сегменту мы обращаемся?

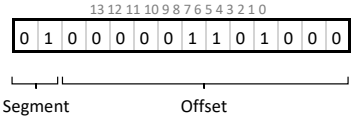
Аппаратное обеспечение использует регистры сегментов во время трансляции. Как узнать смещение в сегмент и к какому сегменту относится адрес?

Один общий подход, иногда называемый **явным** подходом, заключается в разделении адресного пространства на сегменты на основе нескольких верхних битов виртуального адреса; этот метод использовался в системе VAX/VMS. В нашем примере выше у нас есть три сегмента; таким образом, нам нужно два бита для выполнения нашей задачи. Если мы используем два верхних бита нашего 14-битного виртуального адреса для выбора сегмента, наш виртуальный адрес будет выглядеть следующим образом:



В нашем примере, если два верхних бита 00, то аппаратное обеспечение знает, что виртуальный адрес находится в сегменте кода, и, таким образом, использует пару базового и пограничного регистров для кода. Если верхние два бита равны 01, оборудование знает, что адрес находится в куче, и таким образом использует пару базового и пограничного регистров для кучи. Давайте возьмем наш пример виртуального адреса кучи сверху (4200)

и переведем его, чтобы убедиться, что это ясно. Виртуальный адрес 4200, в двоичной форме, можно увидеть здесь:



Как видно из рисунка, два верхних бита (01) указывают оборудованию, к какому `сегменту` мы имеем в виду. Нижние 12 битов `смещение` в этап: 0000 0110 1000, или assembly 0x068, или 104 в десятичном. Таким образом оборудование просто берет первые два бита, чтобы определить, какой регистр сегмента использовать, и затем берет следующие 12 битов как смещение в сегмент. Путем добавления базового регистра к смещению, оборудование добирается до окончательного физического адреса. Обратите внимание, что смещение облегчает проверку границ: мы можем просто проверить, меньше ли смещение границ; если нет, адрес является незаконным. Таким образом, если база и границы были массивами (с одной записью на сегмент), то оборудование будет делать что-то вроде этого, чтобы получить желаемый физический адрес:

```
1   // get top 2 bits of 14-bit VA
2   Segment = (VirtualAddress & SEG_MASK) >> SEG_SHIFT
3   // now get offset
4   Offset = VirtualAddress & OFFSET_MASK
5   if (Offset >= Bounds[Segment])
6   RaiseException(PROTECTION_FAULT)
7   else
8   PhysAddr = Base[Segment] + Offset
9   Register = AccessMemory(PhysAddr)
```

В нашем примере мы можем заполнить значения для констант выше. Конкретно, `SEGMASK` будет установлен на `0x3000`, `SEGSHIFT` на `12`, и `OFFSETMASK` на `0xFFF`.

Возможно, вы также заметили, что, когда мы используем верхние два бита, и у нас есть только три сегмента (код, куча, стек), один сегмент адресного пространства не используется. Таким образом, некоторые системы помещают код в тот же сегмент, что и куча, и, таким образом, используют только один бит для выбора сегмента.

Существуют другие способы для оборудования определить, в каком сегменте находится определенный адрес. В **неявном** подходе аппаратное обеспечение определяет сегмент, замечая, как был сформирован адрес. Если, например, адрес был сгенерирован из счетчика программы (т. е. это была выборка команд), то адрес находится внутри сегмента кода; если адрес основан на стеке или базовом указателе, он должен быть в сегменте стека; любой другой адрес должен быть в куче.

16.3 Что насчет стека?

© 2008–18, ARPACI-DUSSEAU

До сих пор мы не учли один важный компонент адресного пространства: стек. Стек был перемещен на физический адрес 28 КБ на схеме выше, но с одним критическим отличием: он растет в обратном направлении. В физической памяти он начинается с 28 КБ и увеличивается до 26 КБ, что соответствует виртуальным адресам с 16 Кб до 14 Кб; преобразование должно выполняться по-другому.

Первое, что нам нужно, это немного дополнительной аппаратной поддержки. Вместо базовых и граничных значений аппаратное обеспечение также должно знать, в каком направлении растет сегмент (немного, например, значение 1, когда сегмент растет в положительном направлении, и 0 для отрицательного). Наше обновленное представление о том, что аппаратные треки видно на рисунке 16.4.

Segment	Base	Size	Grows Positive?
Code	32K	2K	1
Heap	34K	2K	1
Stack	28K	2K	0

Рис. 16.4: **Сегментные регистры (с поддержкой отрицательного роста)**

С аппаратным пониманием, что сегменты могут вырасти в отрицательном направлении, аппаратные средства должны теперь преобразовать такие виртуальные адреса немного иначе. Давайте возьмем пример виртуального адреса стека и переведем его, чтобы понять процесс.

В данном примере предположим, что мы хотим обратиться к виртуальному адресу 15 КБ, который нужно сопоставить физическому адресу 27 КБ. Наш виртуальный адрес в двоичной форме выглядит так: 11 1100 0000 0000 (hex 0x3C00). Аппаратное обеспечение использует два верхних бита (11) для обозначения сегмента, но затем мы остаемся со смещением 3 КБ. Чтобы получить правильное отрицательное смещение, мы должны вычесть максимальный размер сегмента из 3KB: в этом примере сегмент может быть 4KB, и, таким образом, правильное отрицательное смещение 3KB - 4KB = -1KB. Мы просто добавляем отрицательное смещение (- 1Кб) к основанию (28 КБ), чтобы получить правильный физический адрес: 27 КБ. Проверка границ может быть вычислена путем обеспечения того, что абсолютное значение отрицательного смещения меньше размера сегмента.

16.4 Поддержка обмена

По мере того, как поддержка сегментации росла, разработчики систем вскоре поняли, что они могут реализовать новые типы эффективности с немного большей аппаратной поддержкой. В частности, для экономии памяти иногда полезно **разделить** определенные сегменты памяти между адресными пространствами. В частности, **совместное использование кода** широко распространено и до сих пор используется в системах.

Для поддержки совместного использования, нам нужно немного дополнительной поддержки от оборудования, в виде **protectionbits (защитных битов)**. Базовая поддержка добавляет несколько бит на сегмент, указывая, может ли программа читать из сегмента, или в него писать, или выполнять код, который находится в сегменте. Установив сегмент кода только для чтения, один и тот же код может быть разделен между несколькими процессами, не беспокоясь о вреде изоляции; в то время как каждый процесс все еще думает, что он получает доступ к своей собственной частной памяти, ОС тайно делится памятью, которая не может быть изменена процессом, и, таким образом, иллюзия сохраняется.

Пример дополнительной информации, отслеживаемой аппаратным обеспечением (и ОС), показан на рисунке 16.5. Как видно, сегмент кода настроен на чтение и выполнение, поэтому один и тот же физический сегмент в памяти может быть сопоставлен нескольким виртуальным адресным пространствам.

Segment	Base	Size	Grows Positive?	Protection
Code	32K	2K	1	Read-Execute
Heap	34K	2K	1	Read-Write
Stack	28K	2K	0	Read-Write

Рис. 16.5: **Значения регистра сегмента (с защитой)**

С битами защиты аппаратный алгоритм, описанный ранее, также придется изменить. В дополнение к проверке, находится ли виртуальный адрес в пределах границ, оборудование также должно проверить, допустим ли определенный доступ. Если пользовательский процесс пытается записать в сегмент только для чтения или выполнить из неисполняемого сегмента, оборудование должно вызвать исключение, и таким образом позволить ОС иметь дело с нарушающим процессом.

16.5 Мелкозернистая и крупнозернистая сегментация

Большинство наших примеров до сих пор были сосредоточены на системах с несколькими сегментами (т. е. код, стек, куча); мы можем думать об этой сегментации как о **крупнозернистой**, поскольку она разбивает адресное пространство на относительно большие, грубые куски. Однако некоторые ранние системы (например, Multics) были более гибкими и позволяли адресным пространствам состоять из большого числа меньших сегментов, называемых **мелкозернистой** сегментацией.

Поддержка многих сегментов требует еще большей аппаратной поддержки, так как некоторая таблица сегментов хранится в памяти. Такие **таблицы сегментов** обычно поддерживают создание очень большого числа сегментов и, таким образом, позволяют системе использовать сегменты более гибко, чем мы обсуждали до сих пор. Например, ранние машины, такие как Burroughs B5000, поддерживали тысячи сегментов и

© 2008–18, ARPACI-DUSSEAU

ожидали, что компилятор разделит код и данные на отдельные сегменты, которые ОС и оборудование будут поддерживать. В то время считалось, что, имея мелкозернистые сегменты, ОС может лучше узнать о том, какие сегменты используются, а какие нет, и, таким образом, более эффективно использовать основную память.

16.6 Поддержка ОС

Теперь вы должны иметь общее представление о том, как работает сегментация. Части адресного пространства перемещаются в физическую память по мере работы системы, и, таким образом, достигается огромная экономия физической памяти по сравнению с нашим более простым подходом с помощью всего одной пары регистров база/граница для всего адресного пространства. В частности, все неиспользуемое пространство между стеком и кучей не обязательно должно быть выделено в физической памяти, что позволяет разместить больше адресных пространств в физической памяти.

Однако сегментация порождает ряд новых проблем. Сначала мы опишем новые проблемы ОС, которые необходимо решить. Первый: что делать ОС при переключении контекста? Регистры сегментов должны быть сохранены и восстановлены. Очевидно, что каждый процесс имеет свое собственное виртуальное адресное пространство, и ОС должна убедиться, что правильно настроила эти регистры, прежде чем позволить процессу запуститься снова.

Второй, и более важный, вопрос — это управление свободным пространством в физической памяти. При создании нового адресного пространства ОС должна иметь возможность находить пространство в физической памяти для своих сегментов. Ранее мы предполагали, что каждое адресное пространство имеет одинаковый размер, и поэтому физическую память можно рассматривать как набор слотов, в которые поместятся процессы. Теперь у нас есть несколько сегментов для каждого процесса, и каждый сегмент может быть разного размера.

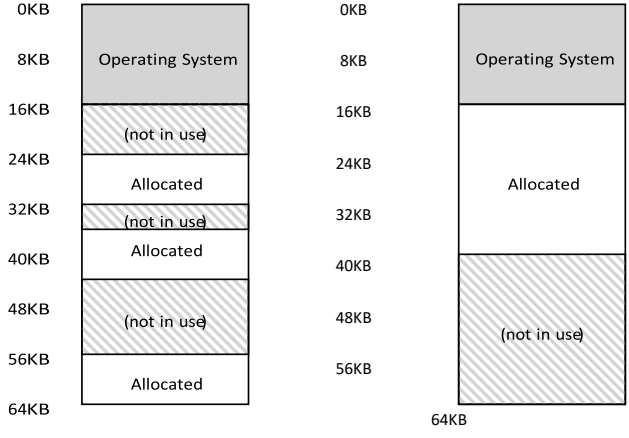


Рис. 16.6: **Неуплотненная и сжатая память**

Главная проблема заключается в том, что физическая память быстро заполняется маленькими “дырками” свободного пространства, что затрудняет выделение новых сегментов или рост существующих. Мы называем эту проблему **внешней фрагментацией**; см. Рисунок 16.6 (слева).

В примере процесс приходит и хочет запросить сегмент 20 КБ. В этом примере есть 24 КБ свободного пространства, но не одним непрерывным сегментом, а тремя несмежными кусками. Таким образом, ОС не может удовлетворить запрос процесса.

Одним из решений этой проблемы может быть **сжатие** физической памяти путем перестановки существующих сегментов. Например, ОС может остановить запущенные процессы, скопировать их данные в одну непрерывную область памяти, изменить значения регистра сегментов, чтобы они указывали на новые физические местоположения, и, таким образом, иметь большой объем свободной памяти для работы. Таким образом, ОС позволяет успешно выполнить новый запрос на выделение. Однако сжатие является дорогостоящим, поскольку копирование сегментов занимает много памяти и обычно занимает достаточно много процессорного времени. Рис. 16.6 (справа) для схема уплотняются физической памяти.

Более простой подход заключается в использовании алгоритма управления свободным списком, который пытается сохранить большие объемы памяти, доступные для выделения. Есть буквально сотни подходов, которые использовали люди, включая такой классический алгоритм как **best-fit (наиболее подходящий)**, в **worst-fit** "первый подходящий", и более сложные схемы, как **buddyalgorithm**. К сожалению, каким бы умным ни был алгоритм, внешняя фрагментация все равно будет существовать; таким образом, хороший алгоритм просто пытается ее минимизировать.

Тот факт, что существует так много различных алгоритмов, пытающихся свести к минимуму внешнюю фрагментацию, свидетельствует о более сильной основополагающей истине: нет одного “лучшего” способа решить проблему. Таким образом, мы соглашаемся на что-то разумное и надеемся, что это достаточно хорошо. Единственное реальное решение (как мы увидим в следующих главах) состоит в том, чтобы полностью избежать проблемы, никогда не выделяя память в блоках переменного размера.

16.7 Краткое изложение.

Сегментация решает ряд проблем и помогает нам построить более эффективную виртуализацию памяти. Помимо динамического перемещения сегментация может лучше поддерживать разреженные адресные пространства, избегая огромной потенциальной траты памяти между логическими сегментами адресного пространства. Это также

© 2008–18, ARPACI-DUSSEAU

быстро, так как выполнение арифметической сегментации легко и хорошо подходит для аппаратного обеспечения; накладные расходы на перевод минимальны. Также возникает дополнительное преимущество: совместное использование кода. Если код помещается в отдельный сегмент, такой сегмент потенциально может быть общим для нескольких запущенных программ.

Однако, как мы узнали, выделение сегментов переменного размера в памяти приводит к некоторым проблемам, которые мы хотели бы преодолеть. Во-первых, как обсуждалось выше, это внешняя фрагментация. Поскольку сегменты являются переменного размера, свободная память измельчается на части нечетного размера, и таким образом удовлетворение запроса выделения памяти может быть трудно. Можно попробовать использовать “умные” алгоритмы или периодически сжимать память, но проблема фундаментальна и ее трудно избежать.

Вторая и, возможно, более важная проблема заключается в том, что сегментация все еще недостаточно гибка для поддержки нашего полностью обобщенного, разреженного адресного пространства. Например, если у нас есть большая, но редко используемая куча в одном логическом сегменте, вся куча должна по-прежнему находиться в памяти для доступа. Другими словами, если наша модель того, как используется адресное пространство, не совсем соответствует тому, как основная сегментация была разработана для ее поддержки, сегментация работает не очень хорошо. Поэтому нам необходимо найти новые решения.